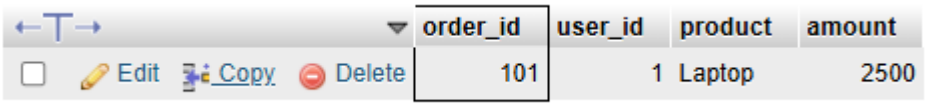
1. Create a SQL script to insert a new order into an Orders table, then use COMMIT to save the transaction and verify that the new order persists after reconnecting to the database.

```
START TRANSACTION;

INSERT INTO Orders (order_id, user_id, product, amount)
VALUES (101, 1, 'Laptop', 2500.00);

COMMIT;
```



	order_id	user_id	product	amount
Edit Copy Delete	101	1	Laptop	2500

2. Simulate a Zomato-style food order process: insert two new items into an OrderItems table, then use ROLLBACK to undo the changes before committing. Check that no new items remain in the table after rollback.

```
START TRANSACTION;

INSERT INTO OrderItems (order_id, product, amount)
VALUES
(101, 'Paneer Pizza', 299.00),
(101, 'Veg Burger', 149.00);

ROLLBACK;

SELECT *
FROM OrderItems
WHERE product IN ('Paneer Pizza', 'Veg Burger');
```

3. In a Flipkart-like shopping cart scenario, use SAVEPOINT to mark a point after adding a product to the Cart table, add another product, then use ROLLBACK TO SAVEPOINT to undo only the last addition. Show the final contents of the Cart table.

```
START TRANSACTION;

-- Add first product
INSERT INTO Cart (product, amount)
VALUES ('iPhone', 50000.00);

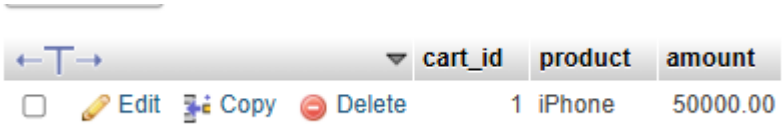
-- Mark savepoint
SAVEPOINT product_added;

-- Add second product
INSERT INTO Cart (product, amount)
VALUES ('Headphones', 2000.00);

-- Undo only the second addition
ROLLBACK TO SAVEPOINT product_added;

-- Save the remaining changes
COMMIT;

-- Show final Cart contents
SELECT * FROM Cart;
```



	cart_id	product	amount
Edit Copy Delete	1	iPhone	50000.00

4. Write a trigger on a Wallet table that automatically deducts the purchase amount from the user's balance whenever a new transaction is inserted, similar to how Paytm updates wallet balance after a payment.

```
DELIMITER //

CREATE TRIGGER deduct_wallet_balance
```

```
AFTER INSERT ON Transactions

FOR EACH ROW

BEGIN

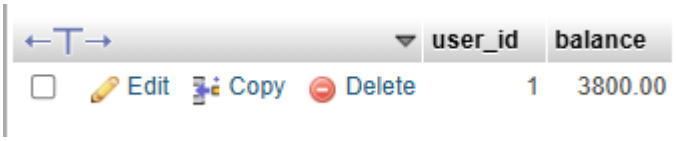
  UPDATE Wallet

  SET balance = balance - NEW.purchase_amount

  WHERE user_id = NEW.user_id;

END //


DELIMITER ;
```



5. Use ChatGPT to generate a SQL test scenario where a trigger incorrectly updates a user's balance after an expense entry. Paste the scenario and your corrected trigger code, explaining how you fixed the error.

Hint: Ask ChatGPT for a buggy trigger example and a test case that exposes the bug.

SQL Trigger Bug and Correction: Expense Entry

1. Buggy Trigger

The following trigger incorrectly adds the expense amount to the user's wallet balance instead of deducting it.

```
DELIMITER //


CREATE TRIGGER update_balance

AFTER INSERT ON Expenses

FOR EACH ROW

BEGIN

  UPDATE Wallet

  SET balance = balance + NEW.amount

  WHERE user_id = NEW.user_id;

END //


DELIMITER ;
```

2. Test Scenario

```
INSERT INTO Wallet (user_id, balance)
VALUES (1, 5000.00);

INSERT INTO Expenses (user_id, amount)
VALUES (1, 1000.00);

SELECT * FROM Wallet;
```

Incorrect Output:

user_id	balance
1	6000.00

The expected balance is ₹4,000, but the buggy trigger updates it to ₹6,000.

3. Corrected Trigger

```
DROP TRIGGER IF EXISTS update_balance;

DELIMITER //

CREATE TRIGGER update_balance
AFTER INSERT ON Expenses
FOR EACH ROW
BEGIN
   UPDATE Wallet
   SET balance = balance - NEW.amount
   WHERE user_id = NEW.user_id;
END //

DELIMITER ;
```

4. Explanation

The error was caused by using the addition operator (+) instead of the subtraction operator (-). I corrected the trigger to subtract NEW.amount from the user's wallet balance whenever a new expense is inserted.

For example, if the initial balance is ₹5,000 and the expense is ₹1,000, the corrected balance will be ₹4,000.

Correct Output:

user_id	balance
1	4000.00